Consejería de Educación, Cultura y Deportes
IES VIRREY MORCILLO
AVDA. MENÉNDEZ PELAYO S/N
967 140 881 02003120.ies@edu.jccm.es

PROGRAMACIÓN ORIENTADA A OBJETOS EN KOTLIN

Manual técnico

Arquitectura, código y mantenimiento del proyecto JuegoRol

Proyecto	JuegoRol · juego de rol por turnos en consola
Repositorio	github.com/ccnl10/JuegoRol
Tecnología	Kotlin 2.1.0 (plugin de Gradle) · Gradle 8.10.2 · JDK 21
Versión del documento	1.0 · octubre de 2026
Destinatarios	Profesorado y alumnado que mantenga o amplíe el proyecto
Documentos relacionados	Manual de usuario · README.md · docs/api/index.html

Índice

1. Introducción 3
1.1 Cifras del proyecto 3
1.2 Entorno 3
2. Estructura del proyecto 5
3. Modelo de clases 6
3.1 Personaje 6
3.2 Héroes y enemigos 7
3.3 Decisiones de diseño 7
4. Descripción de los archivos 8
4.1 Métodos de Partida 8
5. Cómo funciona el combate 9
5.1 Orden de una ronda 9
5.2 Cálculo del daño 9
6. El menú 11
7. Documentación del código 12
7.1 Comentarios KDoc 12
7.2 Página de la API 12
7.3 Generarla con Dokka 12
8. Compilación y distribución 13
8.1 Ejecutar desde IntelliJ 13
8.2 Generar una versión instalable 13
8.3 El wrapper de Gradle 13
9. Cómo ampliar el proyecto 14
10. Pruebas realizadas 15
11. Observaciones 15

1. Introducción

JuegoRol es la solución del ejercicio «Juego de rol por turnos»: un combate en consola entre un equipo de héroes y uno de enemigos. Es un proyecto pequeño, pero reúne casi todo lo visto sobre programación orientada a objetos en Kotlin: clases abstractas, herencia, sobrescritura, interfaces con propiedades y métodos con cuerpo, `enum class`, `data class`, `object`, `companion object`, propiedades con setter personalizado, funciones de extensión y colecciones.

Este documento explica cómo está organizado el código, cómo funciona el combate, por qué se ha diseñado así y cómo compilarlo, documentarlo y ampliarlo. La referencia de cada clase y función está en los comentarios **KDoc** del código y en `docs/api/index.html`.

1.1 Cifras del proyecto

Elemento	Cantidad
Paquetes	1 (`rol`)
Archivos .kt	7 (unas 350 líneas de código, más los comentarios)
Clases, interfaces, enum y objects	13: 1 enum, 2 clases abstractas, 2 interfaces, 6 clases de personaje, 1 data class y 1 object
Miembros públicos documentados	41
Funciones de nivel superior	6 (incluida la función de extensión `barraVida`)
Opciones del menú	7 más Salir

1.2 Entorno

Componente	Versión / configuración	Dónde se define
Sistema de construcción	Gradle 8.10.2 (Kotlin DSL)	`gradle/wrapper/gradle-wrapper.properties`
Kotlin	Plugin `kotlin("jvm")` 2.1.0	`build.gradle.kts`
Aplicación	Plugin `application`, clase principal `rol.MainKt`	`build.gradle.kts`
JDK	Microsoft OpenJDK 21 (ms-21); sirve el 17 o superior	`.idea/misc.xml`
Dependencias	Ninguna externa (solo la biblioteca estándar de Kotlin, de Maven Central)	`build.gradle.kts`
Código fuente	`src/main/kotlin/rol`	Convención de Gradle

> **NOTA · Por qué Gradle**
> A diferencia de un módulo de IntelliJ «puro», con Gradle el proyecto se describe en `build.gradle.kts` y se puede compilar y ejecutar igual en cualquier ordenador, dentro o fuera del IDE. Es el mismo sistema que se usa en Android y en Jetpack Compose. La primera vez que se abre, IntelliJ necesita Internet para descargar Gradle y el plugin de Kotlin.

build.gradle.kts

```
plugins {
    kotlin("jvm") version "2.1.0"
    application
}

group = "rol"
version = "1.0"

repositories {
    mavenCentral()
}

application {
    mainClass.set("rol.MainKt")
}

// Para que readln() funcione también con la tarea run de Gradle
tasks.named<JavaExec>("run") {
    standardInput = System.`in`
}
```

La última instrucción es importante: por defecto, la tarea `run` de Gradle no conecta el teclado con el programa, y `readln()` no recibiría nada. Con `standardInput = System.in` el menú funciona también al ejecutarlo con Gradle.

2. Estructura del proyecto

ÁRBOL DE CARPETAS

```
JuegoRol/
├── README.md                       Presentación del proyecto en GitHub
├── build.gradle.kts                Plugins, clase principal y tarea run
├── settings.gradle.kts             Nombre del proyecto
├── gradle.properties               Estilo de código y consola de Gradle
├── gradle/wrapper/
│   └── gradle-wrapper.properties   Versión de Gradle (8.10.2)
├── docs/
│   ├── api/index.html              Documentación HTML de la API (desde KDoc)
│   ├── img/                        Imágenes del README
│   ├── Manual_Tecnico_JuegoRol.docx / .pdf
│   └── Manual_Usuario_JuegoRol.docx / .pdf
└── src/main/kotlin/rol/
    ├── Personaje.kt                Clase abstracta Personaje y extensión barraVida()
    ├── Arma.kt                     enum class Arma
    ├── Habilidades.kt              Interfaces Sanador y Volador
    ├── Heroes.kt                   Guerrero, Mago y Arquero
    ├── Enemigos.kt                 Enemigo, Goblin, Trol y Dragon
    ├── Partida.kt                  object Partida y data class Pocion
    └── Main.kt                     main(), menú y prueba de las partes 1-4
```

`.gitignore` excluye del repositorio lo que se genera solo: `.gradle/`, `build/`, `out/`, `.idea/` y los `*.iml`. Por eso, al clonar el proyecto, IntelliJ lo reconstruye a partir de los archivos de Gradle.

3. Modelo de clases

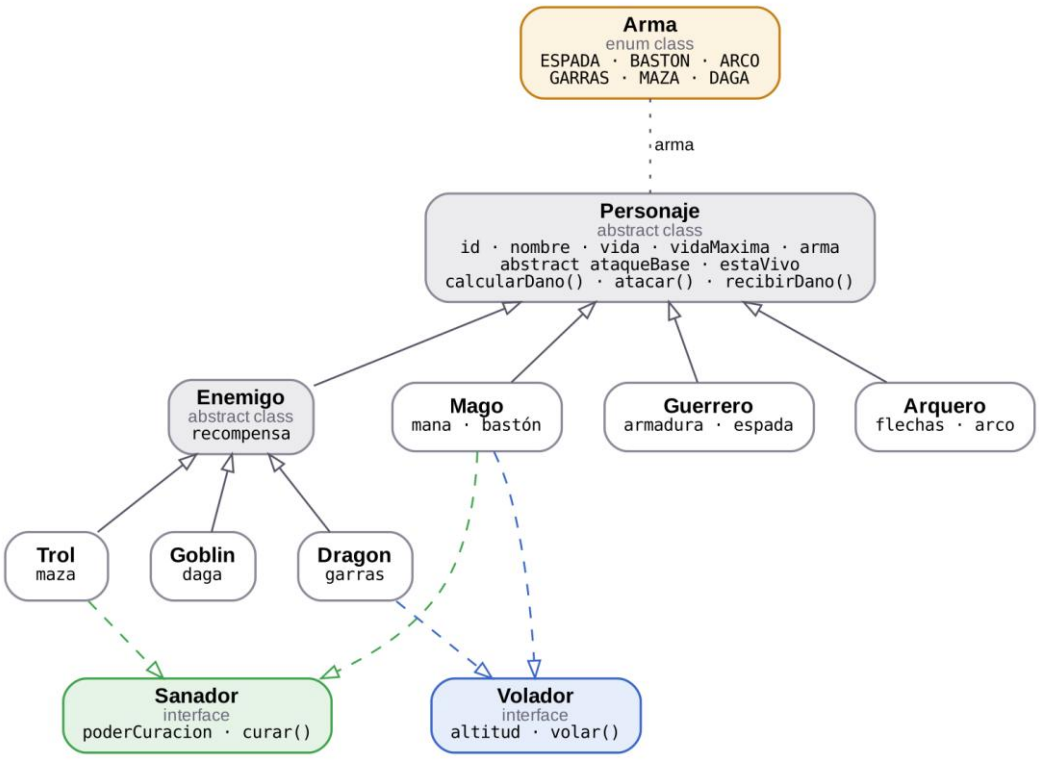


Figura 1. Clases del juego. Flecha continua: hereda. Flecha discontinua: implementa una interfaz. Fondo gris: clase abstracta.

3.1 Personaje

`Personaje` es la raíz de la jerarquía. Guarda lo que tienen todos los personajes y define el comportamiento por defecto del combate:

Miembro	Tipo	Comportamiento
`id`	`val Int`	Identificador único, asignado en `init` con el contador del `companion object`
`nombre`, `vidaMaxima`, `arma`	`val` del constructor	No cambian durante la partida
`vida`	`var Int` con setter	Empieza en `vidaMaxima` y el setter la mantiene siempre entre 0 y `vidaMaxima`
`estaVivo`	`val Boolean` calculada	`vida > 0`; se calcula cada vez que se lee
`ataqueBase`	`abstract val Int`	Cada subclase da el suyo
`calcularDano(objetivo)`	`open fun`	`ataqueBase + arma.dano`
`atacar(objetivo)`	`open fun`	Muestra el ataque y llama a `objetivo.recibirDano(calcularDano(objetivo))`
`recibirDano(cantidad)`	`open fun`	Resta la vida, muestra cuánta queda y avisa si el personaje cae
`toString()`	`override`	`#id nombre (Clase)`

3.2 Héroes y enemigos

Clase	Hereda de	Interfaces	Vida	Ataque	Arma	Particularidad
`Guerrero`	Personaje	—	120	12	espada +8	Armadura (resta daño, mínimo 1); mitad de daño a voladores
`Mago`	Personaje	Sanador, Volador	80	6	bastón +4	Bola de fuego: −10 de maná, doble de daño; cura 25
`Arquero`	Personaje	—	90	9	arco +6	Gasta una flecha por ataque; sin flechas, puñetazo de 2
`Goblin`	Enemigo	—	50	5	daga +3	Recompensa 30 de oro
`Trol`	Enemigo	Sanador	150	10	maza +10	Cura 15; recompensa 150
`Dragon`	Enemigo	Volador	200	18	garras +12	Vuela a 300 m; recompensa 500

3.3 Decisiones de diseño

Decisión	Motivo
`Personaje` y `Enemigo` son abstractas	No se juega con «un personaje» ni con «un enemigo» genérico; `ataqueBase` obliga a cada clase concreta a dar el suyo
`Sanador` y `Volador` son interfaces	Las tienen clases de los dos bandos (`Mago` y `Trol`, `Mago` y `Dragon`) que ya heredan de clases distintas
`Sanador` declara `val nombre`	`curar()` lo necesita para sus mensajes; las clases no lo reescriben porque ya lo heredan de `Personaje`
`curar()` y `volar()` tienen cuerpo	El comportamiento es igual para todos; cada clase solo da `poderCuracion` o `altitud`
`Arma` es un `enum class` con propiedades	Hay un conjunto cerrado de armas, cada una con su texto y su daño
Setter de `vida` con límites	Ningún ataque, curación o poción puede dejar la vida por debajo de 0 ni por encima del máximo
`id` con `companion object`	El contador es de la clase, no de cada personaje; es privado para que nadie lo cambie desde fuera
`Partida` es un `object`	Solo hay una partida y se usa desde cualquier sitio por su nombre
`Pocion` es una `data class`	Se usa como clave del mapa `inventario`: dos pociones con los mismos datos son iguales
`barraVida()` es una función de extensión	Añade presentación a `Personaje` sin meterla dentro de la clase

4. Descripción de los archivos

Archivo	Declaraciones	Papel
Personaje.kt	`abstract class Personaje`, `fun Personaje.barraVida()`	Base de todos los personajes y barra de vida de 10 casillas
Arma.kt	`enum class Arma`	Seis armas con texto y daño
Habilidades.kt	`interface Sanador`, `interface Volador`	Habilidades compartidas por héroes y enemigos
Heroes.kt	`Guerrero`, `Mago`, `Arquero`	Los tres héroes y sus reglas especiales
Enemigos.kt	`abstract class Enemigo`, `Goblin`, `Trol`, `Dragon`	Los enemigos y su recompensa
Partida.kt	`data class Pocion`, `object Partida`	Equipos, rondas, ranking, búsqueda, inventario y estadísticas
Main.kt	`main()`, `mostrarInfo()`, `leerTexto()`, `jugarRondaMenu()`, `pruebaPersonajes()`	Menú y funciones que lo apoyan; prueba de las partes 1 a 4

4.1 Métodos de Partida

Método	Qué hace
`mostrarEquipos()`	Lista los dos equipos con su barra de vida y las etiquetas `[vuela]` y `[cura]`
`jugarRonda()`	Juega una ronda completa (ver apartado 5)
`combateTerminado()`	`true` si han caído todos los héroes o todos los enemigos
`mostrarResultado()`	Victoria con el número de rondas y el oro de los enemigos caídos, o derrota
`ranking()`	Todos los personajes ordenados de más a menos vida
`buscar(nombre)`	Personaje de cualquier equipo por nombre, sin distinguir mayúsculas; `null` si no existe
`usarPocion(heroe, pocion)`	Reto: un héroe vivo bebe una poción; si era la última de su tipo, desaparece del inventario
`mostrarInventario()`	Reto: pociones que quedan y sus unidades
`estadisticas()`	Reto: personajes por clase (y vivos) y vida total de cada equipo

5. Cómo funciona el combate

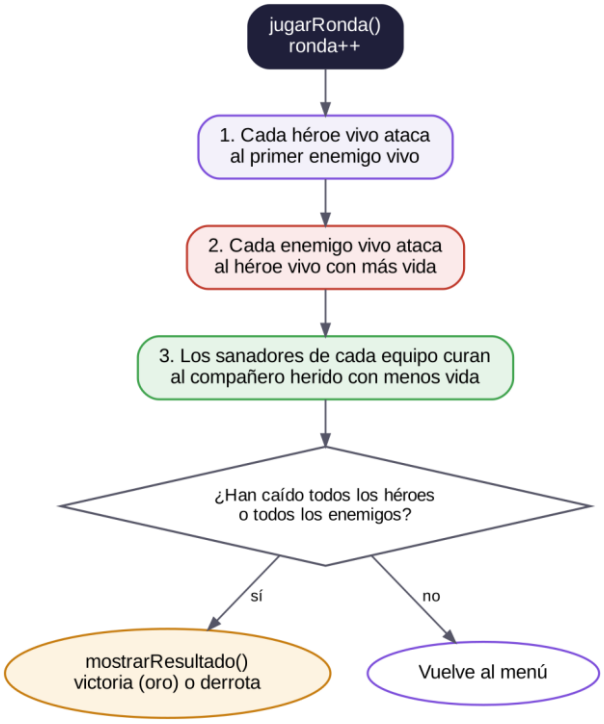


Figura 2. Pasos de una ronda

5.1 Orden de una ronda

1. **Ataque de los héroes.** Cada héroe vivo, en el orden de la lista, ataca al primer enemigo vivo. Si ya no quedan enemigos, el bucle se corta.
2. **Ataque de los enemigos.** Cada enemigo vivo ataca al héroe vivo con más vida (`maxByOrNull`).
3. **Curación.** En cada equipo, cada sanador vivo cura al compañero vivo y herido con menos vida (`minByOrNull`). Un sanador puede curarse a sí mismo, como hace el trol.

5.2 Cálculo del daño

REGLAS

```
daño = ataqueBase + arma.dano                    // Personaje.calcularDano

Guerrero → si el objetivo es Volador: daño / 2
Mago     → si mana >= 10: mana -= 10 y daño * 2 (bola de fuego)
Arquero  → si no quedan flechas: 2 (puñetazo)

Al recibir:
Guerrero → max(daño – armadura, 1)
Resto    → daño completo
vida = vida – daño   (el setter la deja en 0 como mínimo)
```

Ataque	Cálculo	Daño
Guerrero → Goblin	12 + 8	20

Ataque	Cálculo	Daño
Guerrero → Dragón (volador)	(12 + 8) / 2	10
Mago con maná → cualquiera	(6 + 4) × 2	20
Mago sin maná	6 + 4	10
Arquero con flechas	9 + 6	15
Trol → Guerrero con armadura 6	(10 + 10) − 6	14
Dragón → Guerrero con armadura 6	(18 + 12) − 6	24
Dragón → Arquero	18 + 12	30

Así, en la partida del menú (Brunilda con armadura 6, Merlín con 30 de maná y Ariadna con 10 flechas), los héroes ganan en 12 rondas y consiguen 680 de oro.

6. El menú

`main()` forma los equipos y muestra el menú en un bucle `while (true)` que solo termina con `return` al elegir 0. La opción se lee con `readlnOrNull()?.trim()?.toIntOrNull()`, así que un texto o una línea vacía nunca lanzan una excepción: caen en `else` y muestran «Opción no válida.».

Opción	Llama a
1. Ver equipos	`Partida.mostrarEquipos()`
2. Jugar ronda	`jugarRondaMenu(false)`
3. Jugar hasta el final	`jugarRondaMenu(true)`
4. Ver ranking	`Partida.ranking()`
5. Información de un personaje	`mostrarInfo(leerTexto("Nombre: "))`, que usa `Partida.buscar()` e `is` para mostrar los datos de cada tipo
6. Usar poción (reto)	`Partida.mostrarInventario()` y, si quedan, `Partida.usarPocion()`
7. Estadísticas (reto)	`Partida.estadisticas()`
0. Salir	`return` desde `main`

`jugarRondaMenu()` comprueba primero si el combate ya ha terminado. En ese caso no juega más rondas: muestra «El combate ha terminado.» y el resultado.

7. Documentación del código

7.1 Comentarios KDoc

Todas las clases, interfaces, propiedades y funciones públicas tienen un comentario **KDoc**. IntelliJ lo muestra al pasar el ratón por encima (o con **Ctrl + Q**). En las clases, las propiedades del constructor se documentan con `@property`; en las funciones, los parámetros con `@param` y el resultado con `@return`.

EJEMPLO (Habilidades.kt)

```
/**
 * Cura a [objetivo] sumándole [poderCuracion] puntos de vida (sin pasar de su
 * vida máxima). Un personaje que ha caído ya no se puede curar.
 *
 * @param objetivo personaje al que se cura.
 */
fun curar(objetivo: Personaje) {
```

7.2 Página de la API

`docs/api/index.html` reúne todos los KDoc: para cada clase muestra su firma, de quién hereda, sus subclases, sus valores (en el enum) y sus miembros, con buscador y enlaces. Se abre con doble clic en el navegador. Para publicarla en Internet:

1. Sube la carpeta `docs` al repositorio.
2. En GitHub, ve a **Settings** → **Pages**, elige **Deploy from a branch**, la rama principal y la carpeta `/docs`, y pulsa **Save**.
3. En un minuto estará en `https://ccnl10.github.io/JuegoRol/api/`.

7.3 Generarla con Dokka

Como el proyecto usa Gradle, también se puede generar la documentación oficial con **Dokka** añadiendo su plugin:

build.gradle.kts

```
plugins {
    kotlin("jvm") version "2.1.0"
    application
    id("org.jetbrains.dokka") version "2.0.0"
}
```

Después, en el panel **Gradle** de IntelliJ, ejecuta la tarea **Tasks** → **documentation** → **dokkaGenerate**. El resultado queda en `build/dokka/html/index.html`. Con versiones 1.9.x de Dokka la tarea se llama `dokkaHtml`.

8. Compilación y distribución

8.1 Ejecutar desde IntelliJ

Abre `src/main/kotlin/rol/Main.kt` y pulsa el ▶ junto a `fun main()`. También se puede usar el panel **Gradle → Tasks → application → run**.

8.2 Generar una versión instalable

El plugin `application` crea una carpeta con el programa, la biblioteca de Kotlin y scripts de arranque para Windows y Linux:

1. En el panel **Gradle**, ejecuta **Tasks → distribution → installDist**.
2. Se genera `build/install/JuegoRol/`, con las carpetas `bin` y `lib`.
3. Ejecuta `bin\JuegoRol.bat` en Windows o `bin/JuegoRol` en Linux y macOS.

Para entregarlo comprimido, la tarea **distZip** genera `build/distributions/JuegoRol-1.0.zip`, que también se puede adjuntar a una **Release** de GitHub. La tarea `jar` sola no basta, porque el `.jar` resultante no incluye la biblioteca de Kotlin.

8.3 El wrapper de Gradle

> **IMPORTANTE · Faltan los scripts del wrapper**
> El repositorio tiene `gradle-wrapper.properties`, pero no `gradlew`, `gradlew.bat` ni `gradle/wrapper/gradle-wrapper.jar`. IntelliJ funciona igual, porque usa la versión indicada en el archivo de propiedades, pero desde la consola no se puede ejecutar `gradlew run`. Para añadirlos, ejecuta en IntelliJ **Gradle → Tasks → build setup → wrapper** y sube los tres archivos al repositorio.

CON EL WRAPPER YA AÑADIDO

```
gradlew.bat run          (Windows)
./gradlew run            (Linux y macOS)
```

9. Cómo ampliar el proyecto

Añadir un héroe o un enemigo

1. En `Heroes.kt` o `Enemigos.kt`, crea la clase heredando de `Personaje` o de `Enemigo` y pasando vida, arma (y recompensa).
2. Da su `ataqueBase` con `override val`.
3. Si debe curar o volar, implementa `Sanador` o `Volador` y da `poderCuracion` o `altitud`. No hace falta escribir `curar()` ni `volar()`.
4. Si tiene una regla propia, sobrescribe `calcularDano`, `atacar` o `recibirDano` llamando a `super` cuando quieras mantener el comportamiento base.
5. Añádelo a su equipo en `main()`. Si tiene propiedades propias, muéstralas en `mostrarInfo()` con un `if (p is MiClase)`.

Añadir un arma

Añade un valor a `Arma` con su texto y su daño, por ejemplo `HACHA("hacha", 9)`. Como `Arma` no se usa en ningún `when`, no hay que cambiar nada más.

Añadir una habilidad

Crea una interfaz en `Habilidades.kt` siguiendo el modelo de `Volador`: una propiedad abstracta y un método con cuerpo. Después, decide en `Partida.jugarRonda()` en qué momento de la ronda se usa.

Añadir una opción al menú

Escribe la lógica en `Partida` (o en una función de `Main.kt` si es de presentación), añade la línea al menú y una rama al `when` de `main()`.

10. Pruebas realizadas

El proyecto se ha compilado con `kotlinc` 2.0 y se ha ejecutado la partida completa desde el menú y la función `pruebaPersonajes()`. Resultados:

Prueba	Entrada	Resultado esperado	OK
Ver equipos	1	Ids 1 a 6, barras llenas, etiquetas [vuela] y [cura]	✓
Información	5 · merlín	Bastón (+4), vuela a 10 m, cura 25, maná 30	✓
Información	5 · Pepe	No hay ningún personaje con ese nombre.	✓
Ronda 1	2	Snik cae; Brunilda recibe 14 y 24; Merlín la cura hasta 107	✓
Ranking	4	Ignis 200, Grog 150, Brunilda 107…, Snik 0	✓
Poción	6 · Brunilda · pequeña	Vida 120 (no pasa del máximo); quedan 2 pequeñas	✓
Estadísticas	7	6 clases; Goblin con 0 vivos; vida total 290 y 350	✓
Hasta el final	3	Victoria en 12 rondas; oro 680; Ariadna sin flechas en la ronda 11	✓
Opción no válida	9 · abc	Opción no válida.	✓
Prueba partes 1-4	`pruebaPersonajes()`	Guerrero hace 10 al dragón; bola de fuego 20; vida 500 → 150; vida −20 → no vivo; no cura a un caído	✓

11. Observaciones

- Se ha añadido KDoc a todo el código. Solo se han añadido comentarios (y se han sustituido los `//` que repetían lo mismo): el código no cambia.
- Faltan los scripts del wrapper de Gradle (ver apartado 8.3).
- Una poción se gasta aunque el héroe tenga la vida al máximo; la vida no sube porque el setter la limita.
- La partida no se puede reiniciar desde el menú: para jugar otra vez hay que volver a ejecutar el programa.
- Si la entrada estándar se cierra (por ejemplo, al redirigir un archivo sin un 0 final), `readlnOrNull()` devuelve `null` y el menú muestra «Opción no válida.» sin parar. Desde IntelliJ o la consola no ocurre.
- `pruebaPersonajes()` no está en el menú: hay que descomentar una línea de `main()`. Una prueba automática con `kotlin.test` en `src/test/kotlin` sería la forma habitual de comprobarlo sin tocar el código.